

## 8 – SETS & MAPPINGS

### 8.a – Learning Outcomes

By the end of this chapter, you will be able to:

- Define sets and dictionaries and explain their roles as unordered collections in Python.
- Explain the concept of hashability and why sets and dictionary keys require hashable objects.
- Differentiate between hashable and unhashable types (int, float, str, bool, tuple vs. list, dict, set).
- Create sets and dictionaries using appropriate syntax, including `set()` for empty sets and `{}` for empty dictionaries.
- Apply common set methods such as `add()`, `remove()`, `discard()`, `pop()`, and set operations like union, intersection, difference, and symmetric difference.
- Use set operators (`|`, `&`, `-`, `^`) as alternatives to their corresponding methods.
- Construct dictionaries and perform operations including accessing, adding, updating, and deleting key-value pairs.
- Utilize dictionary methods such as `keys()`, `values()`, and `items()` to iterate over dictionary contents.
- Identify the key differences between lists, tuples, sets, and dictionaries in terms of order, mutability, and uniqueness.
- Recognize common pitfalls when working with sets and dictionaries, such as attempting to add mutable objects or relying on element order.

### 8.b – Introduction

Python provides several built-in collection types, each designed for different purposes. While lists and tuples are **sequences** that maintain order and support indexing, sets and dictionaries are **mappings and sets** that use **hash tables** for efficient storage and lookup. Understanding these collections is essential for writing efficient Python programs that handle unique values and key-value relationships.

This chapter covers two important collection types:

- **Sets** – unordered collections of unique, hashable items
- **Dictionaries** – collections of key-value pairs where keys must be hashable

Both rely on the concept of hashability, which we will explore in detail.

## 8.c – Set

A set is an unordered collection of unique items. A set may only contain hashable objects such as:

- int
- float
- str
- bool
- tuples (if all the items are hashable)
- bytes.

### Interesting fact

You may use the `hash()` function to check if an object is hashable.

Sets cannot contain mutable type such as:

- list
- dict
- set
- bytearray

This restriction exists because set elements must be **hashable**, because Python uses hash value for fast lookups and to ensure uniqueness.

### Notes

- Sets do **not** preserve order
- Duplicate values are **automatically removed**
- Set elements must be **hashable**
- Mutable objects like `list` or `dict` cannot be stored in a set.

### 8.c.1 – Creating sets

```
# empty set must use set() not {}
>>> a = set()
>>> a
set()

# set with a single item
>>> b = {0}
>>> b
{0}

# creating sets from other collections
>>> set([1, 2, 3])      #set from a list
```

```
{1, 2, 3}

>>> set((1, 2, 3))      #from a tuple with hashable items
{1, 2, 3}

>>> set({4: 'four', 'five': 5, 6: 'six'})  #from a dict (keys)
{4, 'five', 6}
```

#### 8.c.1.1 – Hashability

A **hash value** is an integer used internally by Python to store objects efficiently in sets and dictionaries. Objects that are **hashable** can be used as set elements or dictionary keys.

```
>>> hash(3)
3
>>> hash('hello')
4800600899085869277
>>> hash([0,9])
Traceback (most recent call last):
  File "<python-input-10>", line 1, in <module>
    hash([0,9])
    ~~~~^^^^^^
TypeError: unhashable type: 'list'
```

List are mutable, so they cannot be hashed.

#### 8.c.2 – Common Set Methods

`add(x)`

Add an element to this set

```
>>> s = {1, 2, 3}
>>> s.add(4)
>>> s
{1, 2, 3, 4}
```

`clear()`

Remove all elements from this set

```
>>> s = {1, 2, 3}
>>> s.clear()
>>> s
{}

```

`copy()`

Create and return a shallow copy of this set

```
>>> s = {1, 2, 'Alice'}
>>> t = s.copy()
>>> t
{1, 2, 'Alice'}

```

`difference(x)`

Return a new set with elements of this set except the ones in the given set arguments.

```
>>> {1, 2, 3}.difference({1, 2})
{3}

```

`difference_update(iter)`

Remove all elements from this set that are members of any of the given set arguments.

```
>>> s = {1, 2, 3}
>>> s.difference_update({1, 2})
{3}

```

`discard(x)`

Remove an element if it exists.

If the element does not exist, nothing happens.

```
>>> s = {'Alice', 'Bob', 'Cloe'}
>>> s.discard('Bob')
>>> s
{'Alice', 'Cloe'}
```

`intersection()`

Return a new set of elements that are members of this and the set argument(s).

```
>>> {1, 2, 3, 4}.intersection({3, 4, 5})
{3, 4}
```

`intersection_update()`

Updates the set so it contains only elements common to both sets.

```
>>> s = {1, 2, 3, 4}
>>> s.intersection_update({3, 4, 5})
>>> s
{3, 4}
```

`isdisjoint(x)`

Return True if their intersection is the empty set.

```
>>> {1, 2, 3, 4}.isdisjoint({'Alice', 'Bob'})
True
```

`issubset()`

Return True if all elements of this set are members of the specified set argument.

```
>>> t = {'Alice', 'Bob', 'Carl', 'Liz'}
>>> {'Alice', 'Bob'}.issubset(t)
True
```

### issuperset()

Return True if all elements of the specified of the set argument are members of this set.

```
>>> {'Alice', 'Bob', 'Carl'}.issuperset({'Alice'})
True
```

### pop()

Remove and return a random element from the set.

Raises `KeyError` if set is empty.

```
>>> s = {'Alice', 'Bob', 'Carl'}
>>> s.pop()
'Alice'
>>> s
{'Bob', 'Carl'}
```

### remove(x)

Remove the specified element from the set.

Raises `KeyError` if the element is not present.

```
>>> s = {'Alice', 'Bob', 'Cloe'}
>>> s.remove('Bob')
>>> s
{'Alice', 'Cloe'}
```

### symmetric\_difference()

Return new set with elements in either this or the specified set argument, but not both.

```
>>> {1, 2, 3}.symmetric_difference({2, 3, 4})
{1, 4}
```

### `symmetric_difference_update()`

Replace this set with the symmetric difference, i.e., elements in either this set or the specified set argument, but not both.

```
>>> s = {1, 2, 3}
>>> s.symmetric_difference_update({2, 3, 4})
>>> s
{1, 4}
```

### `union()`

Return new set with all elements from both sets.

```
>>> {1, 2, 3, 4}.union({3, 4, 5})
{1, 2, 3, 4, 5}
```

### `update()`

Update this set with all elements that are in this or any of the specified set arguments.

```
>>> s = {1, 2, 3, 4}
>>> s.update({3, 4, 5})
>>> s
{1, 2, 3, 4, 5}
```

A sequence is an ordered collection of items that supports operations such as indexing, slicing, iteration and membership testing. They form a core part of the python language and many built-in types are sequences.

Sequences may be categorized into two main groups: Immutable and mutable.

## Set Operators

Python provides operators that correspond to common operations.

| Operators            | Method                              | Operator           | Example                |
|----------------------|-------------------------------------|--------------------|------------------------|
| Union                | <code>union()</code>                | <code> </code>     | <code>A   B</code>     |
| Intersection         | <code>intersection()</code>         | <code>&amp;</code> | <code>A &amp; B</code> |
| Difference           | <code>difference()</code>           | <code>-</code>     | <code>A - B</code>     |
| Symmetric Difference | <code>symmetric_difference()</code> | <code>^</code>     | <code>A ^ B</code>     |

### 8.d – Dictionary

A **dictionary** stores key-value pairs. Keys must be **hashable** (immutable), while values can be any type.

#### Information

The numeric types `int` and `float` are immutable as well as the set type `frozenset`.

```
>>> # Creating dictionaries
>>> empty_dict = {}
>>> grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78}

>>> # Accessing values
>>> grades['Alice']
85

>>> # Adding/updating entries
>>> grades['David'] = 88
>>> grades['Alice'] = 90

>>> # Dictionary methods
>>> grades.keys()
dict_keys(['Alice', 'Bob', 'Charlie', 'David'])

>>> grades.values()
dict_values([90, 92, 78, 88])

>>> grades.items()
dict_items([('Alice', 90), ('Bob', 92), ('Charlie', 78), ('David', 88)])
```

## 8.e – Summary

This chapter introduced sets and dictionaries, two hash-based collections that provide efficient storage and lookup for unique values and key-value relationships.

Key takeaways:

- **Sets** are unordered collections of unique, hashable items. Duplicate values are automatically removed, and sets do not preserve order.
- **Hashability** is required for set elements and dictionary keys. Mutable types (`list`, `dict`, `set`, `bytearray`) cannot be hashed and therefore cannot be used as set elements or dictionary keys.
- **Tuples** are hashable **only if all their elements are hashable**. A tuple containing a list is not hashable.
- **Set operations** include union (`|`), intersection (`&`), difference (`-`), and symmetric difference (`^`), each with corresponding methods.
- **Sets provide  $O(1)$  membership testing**, making them ideal for checking whether an item exists in a collection.
- **Dictionaries** store key-value pairs and use hash tables for  $O(1)$  average-time lookups. Keys must be hashable; values can be any type.
- **Dictionary methods** such as `keys()`, `values()`, and `items()` provide views of the dictionary's contents for iteration.
- **Dictionaries preserve insertion order** (Python 3.7+), though this is an implementation guarantee rather than a fundamental property of the data structure.
- **Shallow copies** of sets can be made with `set.copy()` or `set(old_set)`. The `copy()` method is more readable.

## 8.f – Non coding exercises

1. What is the difference between a list and a tuple?
2. What is the difference between a set and a dictionary?
3. Which collection stores ordered and mutable items.
4. Which collection stores ordered and immutable items.
5. Which collection stores unique values only.
6. Which collection does not allow duplicate values?
7. Which collection stores key-value pairs.
8. Which collection stores key-value pairs?
9. Lists and tuples are identical because both store multiple values. (true or false?)
10. Dictionaries use numeric positions just like lists. (true or false?)

11. Every collection in Python allows duplicate values. (true or false?)
12. List the mutable types in python
13. List the immutable types in python
14. Why can't a list be used as a set element?
15. Why can't a dict be used as a dictionary key?
16. What does `hash()` return for a list? Why?
17. What is the result of `{1, 2, 3} | {3, 4, 5}`?
18. What is the result of `{1, 2, 3} & {3, 4, 5}`?
19. What is the result of `{1, 2, 3} - {3, 4, 5}`?
20. What is the result of `{1, 2, 3} ^ {3, 4, 5}`?
21. Why do sets not preserve order?
22. What is the time complexity of membership testing in a set? Why?
23. Why must dictionary keys be immutable?
24. What happens if you try to use a list as a dictionary key?
25. What is the difference between `remove()` and `discard()` for sets?
26. What is the difference between `pop()` for sets and `pop()` for lists?